

RESUMEN DE MATLAB PARA M&S

¿Cómo inicializar variables en MATLAB?

nombre_de_variable = expresión

Ejemplo: >> **A = 7**

Poner un punto y coma al final de una declaración suprime la salida.

Si lo que queremos es conocer el valor que tiene una variable lo hacemos escribiendo el nombre de la variable y pulsando Intro

Condiciones para nombrar variables:

- El nombre debe comenzar con una letra del alfabeto. Después de eso, el nombre puede contener letras, dígitos y el carácter de subrayado (por ejemplo, valor_l), pero no puede tener un espacio.
- Existe un límite para la longitud del nombre; la función incorporada `namelengthmax` indica cuál es esta longitud máxima (los caracteres adicionales se truncan).
- MATLAB distingue entre mayúsculas y minúsculas. Entonces, las variables llamadas `mynum`, `MYNUM` y `Mynum` son todas diferentes (aunque esto sería confuso y no debería hacerse).
- Aunque los caracteres de subrayado son válidos en un nombre, su uso puede causar problemas con algunos programas que interactúan con MATLAB, por lo que algunos programadores usan mayúsculas y minúsculas en su lugar (por ejemplo, `partWeights` en lugar de `part_weights`).
- Hay ciertas palabras llamadas palabras reservadas, o palabras clave, que no se pueden usar como nombres de variables.
- Los nombres de las funciones integradas pueden, pero no deben, usarse como nombres de variables.

OPERADORES EN MATLAB

Operación	Símbolo	Expresión en Matlab
suma	+	a + b
resta	-	a - b
multiplicación	*	a * b
división	/	a / b
potencia	^	a ^ b

COMANDOS (sin argumentos):

who: muestra las variables que se han definido en esta ventana de comandos (esto solo muestra los nombres de las variables)

whos: muestra las variables que se han definido en esta ventana de comandos (esto muestra más información sobre las variables, similar a lo que está en la ventana del área de trabajo)

clear: borra todas las variables para que dejen de existir

class: puede utilizarse para ver el tipo de cualquier variable (single, double, float, logic, char, etc.)

abs: valor absoluto / variable usada por defecto para almacenar el último resultado

save: guarda el estado de una sesión de trabajo

clc: borramos lo que hay en la ventana de comandos pero no borra las variables de la memoria del espacio de trabajo

quit: cierra MATLAB

grid on: muestra las líneas de cuadrículas principales para los ejes.

hold on: conserva los gráficos en los ejes actuales para que los nuevos gráficos agregados a los ejes no eliminen los gráficos existentes.

clf: elimina todos los elementos secundarios de la figura actual que tienen identificadores visibles

(... ir agregando)

¿Cómo escribir MATRICES?

Igualo el nombre de la matriz, luego abro corchetes y pongo los valores de la primera fila interespaciados, luego pongo un punto y coma y sigo con la segunda fila, y así hasta que termine y cierro corchete. Más abajo algunos ejemplos...

La elipsis, o el operador de continuacion "...", nos deja seguir con la expresión en un renglón nuevo al crear una matriz, como veremos a continuación:

Estas expresiones son equivalentes:

- $A = [12\ 13; 14\ 15]$ // el ";" representa un salto de línea u otra fila de la matriz.
- $A = [12,13; 14,15]$
- $[Y\ I] = \max(A)$
- $[Y,I] = \max(A)$
- $A = [12\ 13$
- $14\ 15]$
- $A = [12\ 13; 14\ 15]$

Si no les pongo " ; " al final, entonces no me lo va a mostrar en la ventana de comandos. Si lo hago me los muestra en la ventana de comandos. Esa es la diferencia

Para referirse a los elementos de una matriz, los subíndices de fila y luego de columna se dan entre paréntesis (siempre la fila primero y luego la columna). Para tomar toda una fila o columna se usa ":". Ejemplo:

- $A(1, :)$ → **Toda la primera fila de la matriz**
- $A(:, 2)$ → **Toda la segunda columna de la matriz**

$A = [a \ b]'$ → traspone la matriz

¿Cómo escribo una función transferencia en MATLAB?

$$H(s) = \frac{2s^3 + s^2 + s + 2}{s^3 + 4s^2 + 5s + 2}$$

Crearé mi variable **num** que representará el numerador de la H(s) y mi variable **den** que representará el denominador. Le debo pasar los coeficientes de los grados de las "s". Siendo el primer valor visto de izquierda a derecha el de mayor grado, y el último valor el de grado 0 (o sea el coeficiente que está solo, sin la s)

```
num = [2 1 1 2];  
den = [1 4 5 2];
```

OBS: Si falta algún grado, recordar poner 0. Por ejemplo, si no tuviera ningún s, tendría $2s^2 + s^2 + 2$ y entonces mi numerador sería **num = [2 1 0 2]**.

Luego creo mi función transferencia:

```
H= tf(num, den);
```

Luego por ejemplo, si quiero pasar esta función de transferencia a un sistema de espacio de estados. Hago:

```
[A, B, C, D] = tf2ss(num, den)
```

¿Cómo escribir un vector de tiempo?

Simplemente hacemos:

```
t = [0: 0.1: 5]
```

Le estamos diciendo que vaya desde 0 hasta 5 segundos y que vaya recorriendo de a 0.1 segundos.

FUNCIONES O COMANDOS CON ARGUMENTOS:

ss: construye un modelo de espacio de estados o convierte modelos LTI a espacio de estado.

Ejemplo de uso:

```
sys = ss(A, B, C, D)
```

Siendo A, B, C y D mis matrices previamente declaradas. Y esto me lo guarda en el objeto **sys**.

tf: transfer function. Esta función se usa para crear la función transferencia.

Ejemplo de uso:

```
sys = tf(num_h, den_h);
```

Le paso como argumento un vector con los coeficientes del numerador y otro vector con los coeficientes del denominador. Y esto me lo guarda en el objeto **sys**.

initial: resuelve el sistema, grafica la respuesta del sistema espacio de estados.

Le debo pasar como argumento el sistema (sys) y la condición inicial (x0)

Ejemplo de uso:

```
[y t x] = initial(system, x0)
```

Lo que va a devolver esta función es: la salida del sistema **y**, el vector de tiempo **t**, que se crea acorde a mi sistema para resolverlo según el objeto, y todas las trayectorias solución que vendría a ser **x**.

plot: dibuja un gráfico

Ejemplo de uso:

```
plot(t, x(:, 1), t, x(:, 2), t, x(:, 3)) → gráfico de soluciones
```

```
plot(x(:,1), x(:,2)) → Diagrama de fase
```

```
plot3(X(:,1), X(:,2), X(:,1).*(-g/l) + X(:,2).*(-k/m), 'Color', 'c', 'LineWidth', 2)
```

Ejemplo de un parcial:

```
plot(x(:,1), x(:,2)) % Diagrama de fase
```

```
xlabel('x1(T)') %Etiquetar el eje de abscisas
```

```
ylabel('x2(T)') % Etiquetar el eje de ordenadas
```

```
title('Plano Fase') % Agregar el título :”Plano Fase”
```

```
plot(t, y)
```

`plot(G, '-or')` %acá usa linea continua (-) simbolo del marcador circular (o) y color rojo (r)

Estilo de Línea

Trazar puntos de datos sin linea

Si especifica un marcador, pero no un estilo de linea, solo se trazarán los marcadores. Por ejemplo:

```
plot(x,y,'d')
```

Especificadores de estilo de linea

Se indican los estilos de linea, los tipos de marcadores y los colores que se desea mostrar, detallados en las tablas siguientes:

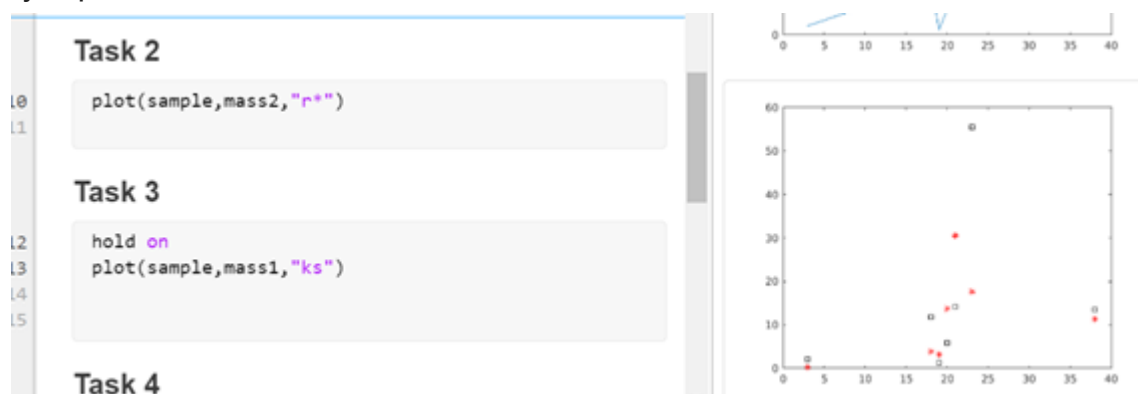
Especificador	Estilo de linea
'-'	Línea continua (valor predeterminado)
'--'	Línea de guiones
'.'	Línea de puntos
'-.'	Línea de guiones y puntos

Estilo de color

Especificadores de color

Especificador	Color
r	Rojo
g	Verde
b	Azul
c	Cian
m	Magenta
y	Amarillo
k	Negro
w	Blanco

Ejemplo



subplot: divide la ventana de gráfico en secciones

subplot(m,n,p) divide la figura actual en una cuadrícula de m por n y crea ejes en la posición especificada por p.

Ejemplo de uso:

`subplot(1, 2, 1)`

El primer número (1) me habla de la figura. Luego cuantos objetos va a haber en la figura (2) y qué posición me va a ocupar (1).

legend: agrega la leyenda al gráfico

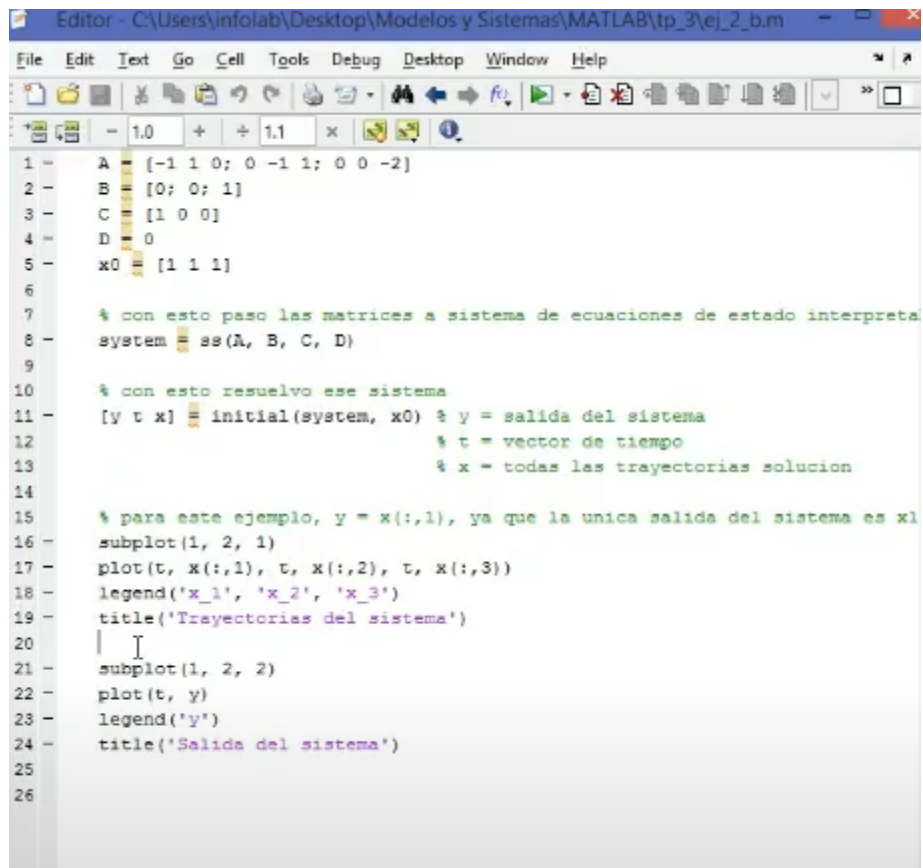
Ejemplo de uso:

`legend(' y ')`

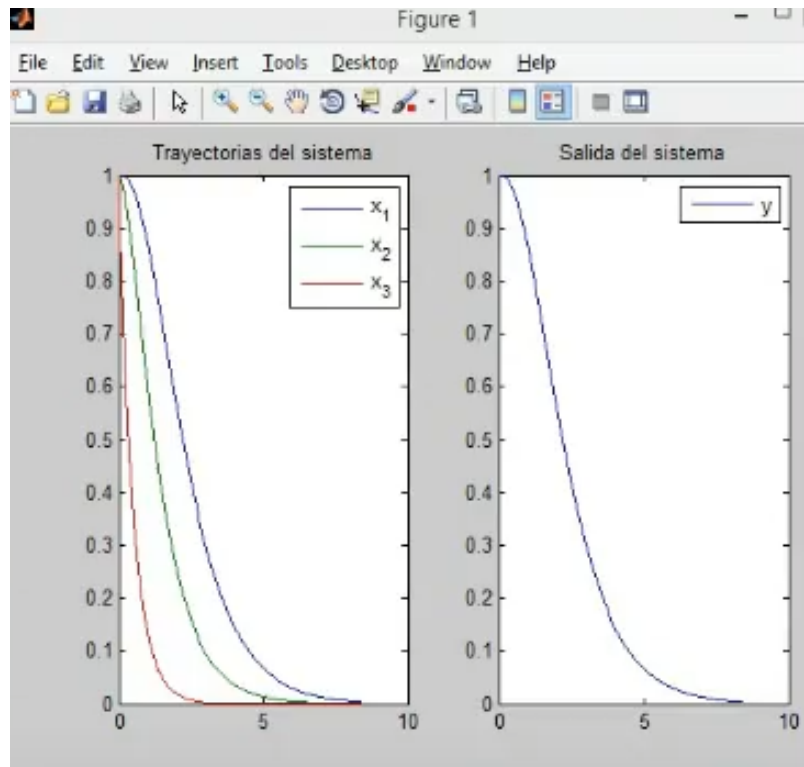
title: agrega un título al gráfico actual

Ejemplo de uso:

`title(' Trayectorias del sistema ')`



```
1 - A = [-1 1 0; 0 -1 1; 0 0 -2]
2 - B = [0; 0; 1]
3 - C = [1 0 0]
4 - D = 0
5 - x0 = [1 1 1]
6
7 - % con esto paso las matrices a sistema de ecuaciones de estado interpreta
8 - system = ss(A, B, C, D)
9
10 - % con esto resuelvo ese sistema
11 - [y t x] = initial(system, x0) % y = salida del sistema
12 -                               % t = vector de tiempo
13 -                               % x = todas las trayectorias solucion
14
15 - % para este ejemplo, y = x(:,1), ya que la unica salida del sistema es x1
16 - subplot(1, 2, 1)
17 - plot(t, x(:,1), t, x(:,2), t, x(:,3))
18 - legend('x_1', 'x_2', 'x_3')
19 - title('Trayectorias del sistema')
20 -
21 - subplot(1, 2, 2)
22 - plot(t, y)
23 - legend('y')
24 - title('Salida del sistema')
25
26
```



xlabel/ylabel: agrega una identificación al eje horizontal/vertical del gráfico actual

Ejemplo de uso:

```
xlabel('Real')
ylabel('Imaginary')
```

lsim: simula un sistema lineal

`lsim(sys, u, t)` traza el tiempo de respuesta simulado del modelo de sistema dinámico `sys` al historial de entrada `(t, u)`. El vector `t` especifica las muestras de tiempo para la simulación. Para sistemas de entrada única, la señal de entrada `u` es un vector de la misma longitud que `t`. Para sistemas de múltiples entradas, `u` es una matriz con tantas filas como muestras de tiempo (longitud `(t)`) y tantas columnas como entradas haya en `sys`.

`y = lsim(sys, u, t)` devuelve la respuesta del sistema `y`, muestreada en los mismos tiempos `t` que la entrada. Para sistemas de salida única, `y` es un vector de la misma longitud que `t`. Para sistemas de múltiples salidas, `y` es una matriz que tiene tantas filas como muestras de tiempo (longitud `(t)`) y tantas columnas como salidas haya en `sys`. Esta sintaxis no genera un gráfico.

dlsim: simulación de sistemas lineales de tiempo discreto

size: devuelve la dimensión de un vector o una matriz

Ejemplo de uso:

```
[f, c] = size(h);
```

step: dibuja la respuesta al escalón (unitario)

Ejemplo de uso:

```
[h, t] = step(H)
```

devuelve un vector de tiempos t correspondiente a las respuestas en h

dstep: respuesta al escalón de sistemas lineales de tiempo discreto

bode: dibuja el diagrama de Bode

```
[modulo, fase, w] = bode(H)
```

H almacena la función transferencia. La variable "modulo" almacena el modulo de la transferencia del sistema. La "fase" almacena la diferencia de fase entre entrada y salida. Todo esto en cada una de las frecuencias "w". Las frecuencias son automáticamente determinadas por la función, dependiendo la dinámica del sistema.

ss2tf: representación espacio de estados a función de transferencia

Ejemplo de uso:

```
[num den] = ss2tf(A, B, C, D)
```

tf2ss: función de transferencia a representación espacio de estados

Ejemplo de uso:

```
[A, B, C, D] = tf2ss(num, den)
```

Siendo **num** el vector con los coeficientes del numerador de la función transferencia, y **den** el vector denominador.

impulse: respuesta al impulso de sistemas lineales de tiempo continuo

Ejemplo de uso:

```
impulse(H)
```

```
impulse(sys)
```

meshgrid: transforma el dominio especificado por un vector único o dos vectores x e y en matrices X e Y con el objeto de usarlas en la evaluación de funciones de dos variables. Las filas de X son copias del vector x y las columnas de Y son copias del vector y

```
[X1,X2] = meshgrid(x1);
```

```
[X,Y] = meshgrid(x, y) %devuelve coordenadas de cuadrícula 2-D basadas en las coordenadas contenidas en los vectores x e y
```


X es una matriz donde cada fila es una copia de x, e Y es una matriz donde cada columna es una copia de y. La cuadrícula representada por las coordenadas X e Y tiene filas de longitud (y) y columnas de longitud (x).

series: interconexión en serie de sistemas lineales que no dependan del tiempo

Ejemplo de uso:

```
sys = series(sys1, sys2)
```

roots: calcula las raíces de un polinomio.

Ejemplo de uso:

```
roots([1 2 0 1])
```

Recibe como argumento el vector de coeficientes del polinomio, empezando por el mayor grado, de izquierda a derecha, hasta el de grado 0.

`r = roots(p)` devuelve las raíces del polinomio representado por p como vector columna. La entrada p es un vector que contiene los coeficientes del polinomio $n+1$, empezando por el coeficiente de x^n . Un coeficiente de 0 indica una potencia intermedia que no está presente en la ecuación. Por ejemplo, $p = [3 \ 2 \ -2]$ representa el polinomio $3x^2 + 2x - 2$.

La función roots resuelve las ecuaciones polinomiales con el formato

$p_1x^n + \dots + p_nx + p_{n+1} = 0$. Las ecuaciones polinomiales contienen una sola variable con exponentes no negativos.

scatter: `scatter(x, y)` crea un diagrama de dispersión con marcadores circulares en las ubicaciones especificadas por los vectores x e y.

zplane: `zplane(z,p)` representa los ceros especificados en el vector columna z y los polos especificados en el vector columna p en la ventana de figura actual. El símbolo 'o' representa un cero y el símbolo 'x' representa un polo. La gráfica incluye el círculo de la unidad como referencia.

Recibe como argumento los ceros y los polos de una función que suele ser la de transferencia.

freqz: respuesta de frecuencia del filtro digital

Ejemplo de uso:

```
freqz(N, 1)
```

fft: transformada de fourier rápida

Ejemplo de uso:

`fft(N)`

Siendo N el vector de coeficientes de polinomios del numerador de la función transferencia

conv: hace la convolución

`conv(h, x)`

Siendo h y x vectores, h es el vector de coeficientes z de exponente negativo

Ejemplo:

$h = [0.5; 1; 0.5] \rightarrow$ el primer 0.5 pertenece a n, el 1 al n-1 y el último a n-2 y así...

$x = [1; 2; 3; 0; -1]$

Entonces $H(z) = 0.5 + z^{-1} + 0.5 z^{-2}$ y además $y(n) = 0.5 \cdot X(n) + X(n-1) + 0.5 \cdot X(n-2)$

stem: traza la secuencia de datos, Y, en los valores especificados por X. Las entradas X e Y deben ser vectores o matrices del mismo tamaño. Además, X puede ser un vector de fila o columna e Y debe ser una matriz con filas de longitud (X).

Ejemplos de uso:

`stem(Y)` traza la secuencia de datos Y como raíces que se extienden desde valores igualmente espaciados y generados automáticamente a lo largo del eje x. Cuando Y es una matriz, la raíz grafica todos los elementos en una fila con el mismo valor de x.

`stem(X, Y)` grafica X versus las columnas de Y. X e Y son vectores o matrices del mismo tamaño. Además, X puede ser un vector de fila o columna e Y una matriz con filas de longitud (X).

impz: devuelve la respuesta al impulso del filtro digital con coeficientes de numerador "b" y coeficientes del denominador "a" de la función transferencia. La función elige el número de muestras y devuelve los coeficientes de respuesta en "h" y los tiempos de muestra en "t".

Ejemplo de uso:

`(h, t) = impz(b, a)`

filter: me pide la función transferencia, (nuevamente, los coeficientes), numerador b y denominador a. Además pide una entrada de datos (un impulso), por ejemplo un delta de Kronecker al estar en tiempo discreto. Me devuelve la respuesta al filtro.

Ejemplo de uso:

```
h = filter(b,a,x)
```

pzmap: devuelve/calcula los polos y los ceros de mi función transferencia como vectores de columna "p" y "z". El gráfico de polo cero no se muestra en la pantalla.

Ejemplo de uso:

```
[p,z] = pzmap(h);
```

zeros: crea una matriz de todos ceros.

Ejemplo de uso:

```
delta = zeros(n); %crea una matriz de nxn  
delta = zeros(10, 1); %crea una matriz de 10x1 de todos ceros  
delta = zeros(n); %crea una matriz de nxn
```

polarplot: traza una línea en coordenadas polares, con theta indicando el ángulo en radianes y rho indicando el valor del radio para cada punto.

Las entradas deben ser vectores de igual longitud o matrices de igual tamaño. Si las entradas son matrices, entonces polarplot traza columnas de rho versus columnas de theta.

```
polarplot(theta,rho)
```

Ejemplo de ecuación diferencial:

```
f = @(t, x) [( x(1) - 2*x(2))^2 ; 3*x(1)-x(2)];
```

Pasos para resolver un sistema en Matlab:

1. Declarar las constantes que tengo con sus respectivos valores
2. Cargo los coeficientes de las variables de estados, simplificandolos en cuanto sea posible.
3. Construyo las matrices del sistema usando los coeficientes declarados en el punto anterior.
4. Genero numerador y denominador de H(s)
5. Genero H(s)

ODE:

Higher-Order ODEs

`[t, x] = ode45(F, t, x0);` %toma como argumento la función, definida anteriormente como ec. diferencial, el tiempo t y la condición inicial x0. Devuelve las soluciones x del sistema a tiempo t.

El MATLAB ODE solo resuelven ecuaciones de primer orden. Debe reescribir las EDO de orden superior como un sistema equivalente de ecuaciones de primer orden utilizando sustituciones genéricas

$$y_1 = y$$

$$y_2 = y'$$

$$y_3 = y''$$

⋮

$$y_n = y^{(n-1)}.$$

El resultado de estas sustituciones es un sistema de n ecuaciones de primer orden

$$y_1' = y_2$$

$$y_2' = y_3$$

⋮

$$y_n' = f(t, y_1, y_2, \dots, y_n).$$

Por ejemplo, considere la EDO de tercer orden

$$y''' - y'' + 1 = 0.$$

Usando la sustitución

$$y_1 = y$$

$$y_2 = y'$$

$$y_3 = y''$$

da como resultado el sistema equivalente de primer orden

$$y_1' = y_2$$

$$y_2' = y_3$$

$$y'_3 = y_1 y_3 - 1.$$

El código para este sistema de ecuaciones sería:

```
function dydt = f(t,y)
    dydt(1) = y(2);
    dydt(2) = y(3);
    dydt(3) = y(1)*y(3)-1;
end
```

Stepinfo

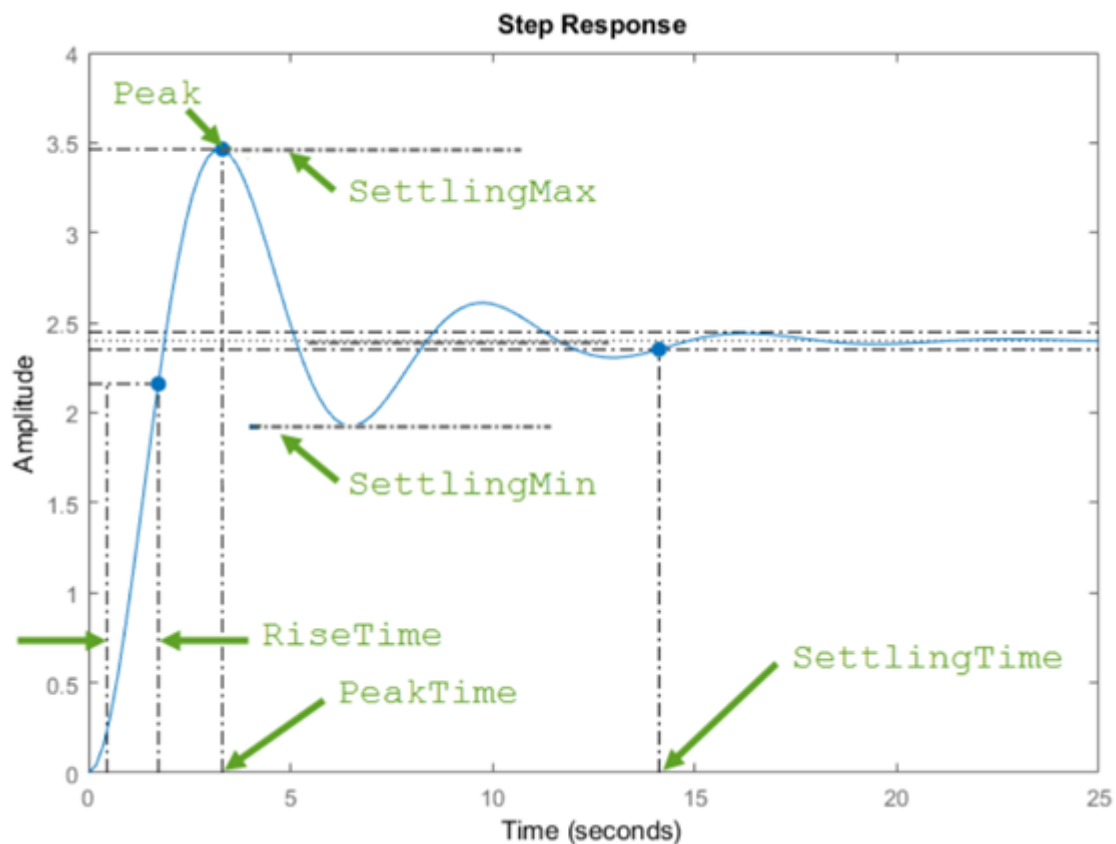
`S = stepinfo (sys)` calcula las características de respuesta al escalón para un modelo de sistema dinámico `sys`. Por defecto, `stepinfo` define el tiempo de establecimiento como el tiempo que tarda el error $e(t) = |y(t) - y_{\text{final}}|$ en caer por debajo del 2% del valor pico de $e(t)$. Además, `stepinfo` define el tiempo de subida como el tiempo que tarda la respuesta en subir del 10% de y_{final} al 90%. Estas definiciones se pueden cambiar usando *SettlingTimeThreshold* y *RiseTimeThreshold*.

Argumento de entrada `SYS` Sistema dinámico a partir de un modelo. El sistema puede ser SISO o MIMO. En nuestro caso usamos modelos LTI numéricos de tiempo continuo o discreto, como los modelos **tf**, **ss**.

La función devuelve las características en una estructura que contiene los campos:

- **RiseTime**: tiempo que tarda la respuesta en aumentar del 10% al 90% de la respuesta de estado estable.
- **SettlingTime**: tiempo que tarda el error $e(t) = |y(t) - y_{\text{final}}|$ entre la respuesta $y(t)$ y la respuesta de estado estable y_{final} para caer por debajo del 2% del valor pico de $e(t)$.
- **SettlingMin**: valor mínimo de $y(t)$ una vez que ha aumentado la respuesta.
- **SettlingMax**: valor máximo de $y(t)$ una vez que ha aumentado la respuesta.
- **Sobreimpulso**: porcentaje de sobreimpulso, relativo a y_{final} .
- **Subimpulso**: porcentaje de subimpulso.
- **Peak** - Valor absoluto máximo de $y(t)$
- **PeakTime**: tiempo al que se produce el valor máximo.

Si sys es inestable, entonces todas las características de respuesta al escalón son NaN, excepto Peak y PeakTime, que son Inf.

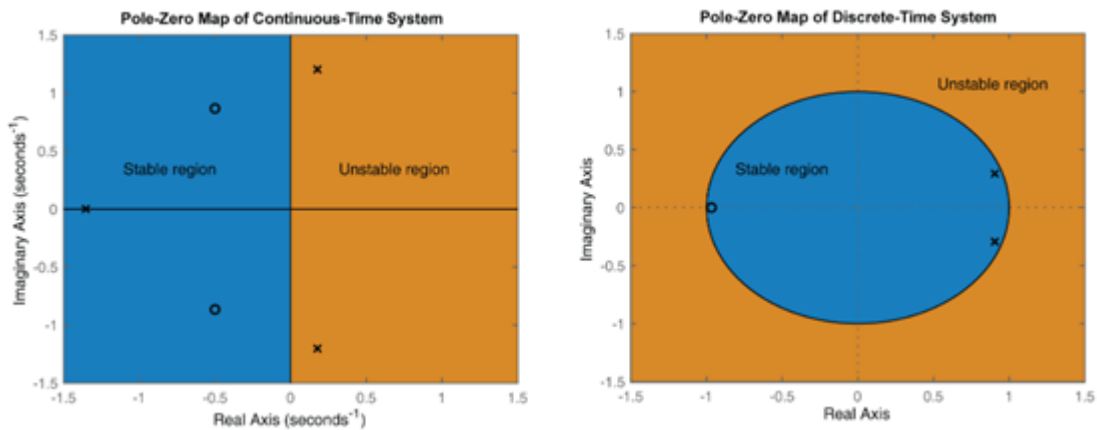


Step `step (sys)` grafica la respuesta de un modelo de sistema dinámico a una entrada escalon de amplitud unitaria. El modelo `sys` puede ser de tiempo continuo o discreto, y SISO o MIMO. Para los sistemas MIMO, el gráfico muestra las respuestas de paso para cada canal de E / S. `step` determina automáticamente los pasos de tiempo y la duración de la simulación en función de la dinámica del sistema.

Damp `damp (sys)` muestra la relación de amortiguación, la frecuencia natural y la constante de tiempo de los polos del modelo lineal `sys`. Para un modelo de tiempo discreto, la tabla también incluye la magnitud de cada polo. Los polos se clasifican en orden creciente de valores de frecuencia.

ejemplo `[wn, zeta] = damp (sys)` devuelve las frecuencias naturales `wn` y las relaciones de amortiguacion `zeta` de los polos de `sys`.

pzmap pzmap (sys) crea un gráfico de polos y ceros del modelo de sistema dinámico continuo o de tiempo discreto sys. "x" y "o" indican los polos y ceros respectivamente, como se muestra en la siguiente figura.



bode bode (sys) crea un diagrama de Bode de la respuesta de frecuencia de un modelo de sistema dinámico sys. El gráfico muestra la magnitud (en dB) y la fase (en grados) de la respuesta del sistema en función de la frecuencia. bode determina automáticamente las frecuencias para graficar basándose en la dinámica del sistema.

[mag, fase, wout] = bode (sys) devuelve la magnitud y la fase de la respuesta en cada frecuencia en el vector wout. La función determina automáticamente las frecuencias en wout basándose en la dinámica del sistema. Esta sintaxis no grafica el diagrama.

Subplot: crear ejes en posición de mosaico. subplot(m,n,p) divide la figura actual en una cuadrícula de m por n y crea ejes en l